

Project Requirements Specification (PRS)

Project Title: LangChain Chatbot using RunnableBranch & RunnableParallel

Version: 1.0

1. Project Objective

Develop an AI chatbot using **LangChain** that demonstrates the implementation of:

- RunnableBranch
- RunnableParallel
- Pydantic Structured Output

The application must provide a **Streamlit-based chat interface** and be implemented using the **latest LangChain APIs**.

2. Functional Requirements

The application shall satisfy the following requirements.

FR-1: Chat Model

The chatbot shall:

- Use any supported Large Language Model (LLM), such as:
 - ChatGroq
 - OpenAI
 - Gemini
 - HuggingFace
- Store API keys securely using a `.env` file.

FR-2: PromptTemplate

The chatbot shall:

- Use `PromptTemplate` for prompt construction.
- Accept dynamic user input through prompt variables.
- Avoid writing prompts directly inside `invoke()`.

FR-3: Pydantic Structured Output (Mandatory)

The chatbot shall:

- Define a response schema using Pydantic (`BaseModel`).
- Generate outputs that conform to the defined schema.
- Display the structured output within the Streamlit application.

The schema may include fields such as:

- answer
- summary
- confidence
- category
- keywords

Using only `StrOutputParser` does **not** satisfy this requirement.

FR-4: RunnableBranch (Mandatory)

The chatbot shall use `RunnableBranch` to execute different prompt pipelines based on user input.

Example routing strategies include:

- Programming → Programming Assistant
- Mathematics → Math Tutor
- General Queries → General Assistant

Students may implement their own branching logic.

FR-5: RunnableParallel (Mandatory)

The chatbot shall use `RunnableParallel` to generate multiple outputs simultaneously from a single user request.

Possible parallel outputs include:

- Answer + Summary
- Answer + Keywords
- Answer + Follow-up Questions
- Answer + Difficulty Level

Students may choose any meaningful parallel outputs.

FR-6: Streamlit User Interface

The application shall provide a Streamlit interface containing:

- Chat interface
- Chat history
- User input box
- AI response display

The following feature is optional:

- Clear Chat button

3. Suggested Project Structure

```
project/
├── app.py
├── chatbot.py
├── prompts.py
├── schemas.py
├── requirements.txt
├── .env.example
├── README.md
└── assets/
```

4. Documentation Requirements

The repository shall include a `README.md` containing:

- Project overview
 - Features
 - RunnableBranch implementation
 - RunnableParallel implementation
 - Pydantic Structured Output implementation
 - Installation instructions
 - Screenshots (optional)
-

5. Submission Requirements

Students shall submit **only** the GitHub repository URL.

The repository must include:

- Complete source code
- README.md
- requirements.txt
- `.env.example`

The following shall **not** be uploaded:

- `.env`
 - API keys
 - Sensitive credentials
-

6. Evaluation Criteria

Component	Marks
PromptTemplate	10
Pydantic Structured Output	20
RunnableBranch Implementation	25
RunnableParallel Implementation	25
Streamlit User Interface	10
Code Quality, Project Structure & Documentation	10
Total	100

7. Constraints

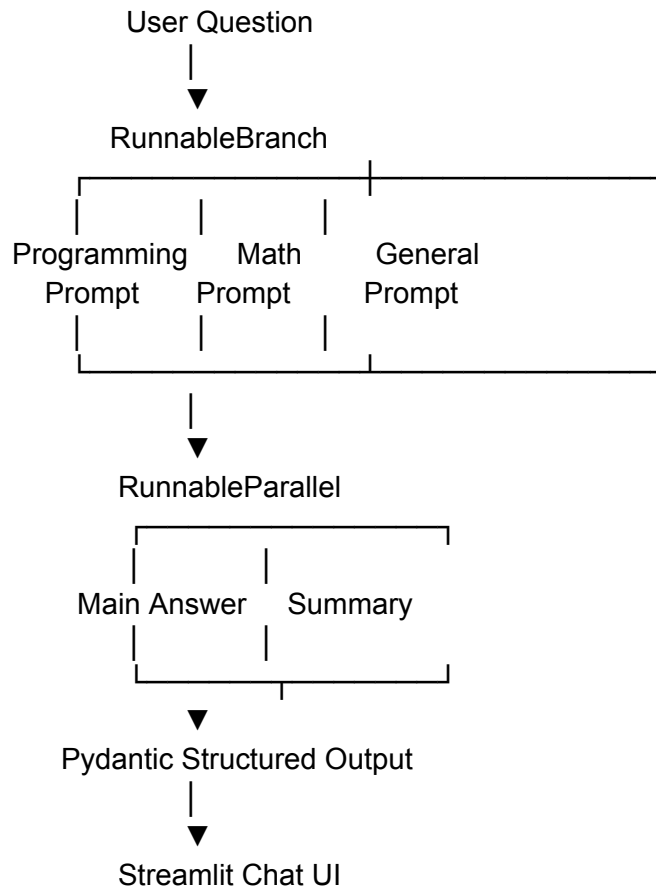
Students shall:

- Use the latest LangChain APIs.
- Implement RunnableBranch.
- Implement RunnableParallel.
- Use Pydantic Structured Output.
- Follow clean coding practices.
- Organize the project into logical modules.

Students shall **not**:

- Copy projects from GitHub, YouTube, or other sources.
 - Use deprecated LangChain APIs.
 - Hardcode API keys.
 - Upload the `.env` file.
 - Submit ZIP files or Google Drive links.
-

8. Example Workflow



9. Acceptance Criteria

The project will be considered complete only if:

- RunnableBranch is correctly implemented.
- RunnableParallel is correctly implemented.
- Pydantic Structured Output is used for response validation.
- The chatbot is accessible through a Streamlit interface.
- The GitHub repository contains the required files and documentation.
- The project follows the latest LangChain syntax and demonstrates good software engineering practices.

